

MANTIS — Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—Multi-agent systems built on large language models (LLMs) are increasingly deployed in banking for transaction monitoring, operations planning, and end-of-day reconciliation, yet there is no standard, reusable way to evaluate such systems under adversarial conditions without either rewriting the framework core or the business workflow logic being tested. We present MANTIS (Modular Agent Network Testbed for Instrumentation and Security), a configuration-driven testbed built around a banking multi-agent system spanning front-, mid-, and back-office workflows. MANTIS instruments five fixed control points (input, agent, interaction, tool, and output) through a hook bus that lets an experimenter declare an attack or failure condition entirely in YAML, register it as a plugin against the same interface used by every other plugin, and leave the underlying banking agents unmodified. Every run produces a signed manifest (configuration hash, seed, and environment metadata), a complete OpenTelemetry- and MLflow-integrated trace, and an automated evaluation of tool-use correctness and terminal-state outcome. We describe the architecture, the banking runtime adapter covering 31 agents and 19 tools across three domains, four concrete attack plugins with machine-readable ground truth, and the observability and evaluation pipeline. We report preliminary, live-verified evidence that all five control points and four attack plugins behave as intended. A zero-latency mock model and a volume-scaling benchmark mode, both implemented to isolate instrumentation overhead from confounding factors, revealed that our initial hypothesis was incomplete: removing LLM sampling latency alone did not resolve measurement noise, since per-process startup cost dominates at the sample sizes tested – an actionable, if unexpected, finding that sharpens what a full statistical evaluation campaign will need to control for.

Index Terms—multi-agent systems, adversarial security, large language models, banking, observability, benchmarking, red teaming

I. Introduction

Large language model (LLM) agents are moving from single-turn assistants into orchestrated multi-agent systems that carry out multi-step business processes. In banking specifically, recent work has proposed multi-agent frameworks for anomaly detection in financial markets [1] and for credit assessment [2], each decomposing a task across specialized agents – a pattern that mirrors how a bank actually organizes front-, mid-, and back-office work today.

That same decomposition is also where new security risk enters. Safety properties established for a single chat model do not transfer to a system where behavior emerges from message passing, tool invocation, and routing decisions across multiple agents [5]. Recent attacks demonstrate that inter-agent communication itself is an exploitable surface: prompt injection can propagate from one agent to another like an infection [3], and an adversary positioned between two agents can intercept and rewrite the messages passing between them without ever touching either agent’s model weights or prompts [4]. A survey of agentic security threats and defenses concludes that most current defenses evaluated in isolation can be bypassed by an adaptive attacker [5], underscoring that security evaluation for these systems needs to happen at the level of the deployed multi-agent system, not the underlying model.

Despite this, there is no widely available testbed that lets a researcher take a realistic, domain-specific multi-agent system and run controlled adversarial experiments against it without either reimplementing the orchestration framework or editing the business logic under test. General-purpose red-teaming benchmarks for LLM agents

exist [7], but they are typically domain-agnostic and are not designed to instrument an existing, unmodified business workflow at fixed points chosen for their security relevance.

This paper presents MANTIS, a modular, configuration-driven testbed built directly on top of a banking multi-agent system spanning front-office (transaction monitoring, fraud review, compliance, customer service), mid-office (operations planning, representative assist), and back-office (end-of-day reconciliation and settlement) workflows. Our contributions are:

- A hook bus architecture that exposes five fixed control points – input, agent, interaction, tool, and output – around an existing agent orchestration pipeline, so that an attack or failure plugin can observe, mutate, delay, or block a single interaction without any change to the banking agents themselves.
- A banking runtime adapter covering three domains, 31 agents, and 19 real banking tools (customer lookup, transaction execution, ledger posting, reconciliation, and reporting), introspected live from the running system rather than maintained as a separate, driftable inventory.
- Four attack plugins – prompt injection, inter-agent message spoofing, route confusion, and tool-parameter mutation – registered through a single plugin interface and carrying machine-readable ground truth (expected tools, forbidden tools, and expected terminal state) for automated grading.
- A standard observability pipeline that maps every hook dispatch to a typed trace event (experiment, workflow, agent, interaction, tool, security, and evaluation families), exports to OpenTelemetry and MLflow, and writes portable JSONL traces annotated with banking-specific semantics such as risk level, data sensitivity, and financial side effect.
- A reproducible evaluation and benchmarking CLI that hashes every configuration, records environment metadata (interpreter version, dependency versions, and source commit) alongside each run, scores trace completeness and tool-use correctness, and sweeps a directory of experiment configurations into a single comparative report.

The remainder of this paper is organized as follows. Section II motivates the problem and situates MANTIS relative to prior work on agentic security and Zero Trust access control. Section III describes the MANTIS architecture in detail. Section IV reports preliminary, live-verified evidence for each component and describes the full evaluation campaign this testbed is designed to run. Section V concludes and outlines future work.

II. Background and Motivation

A. Multi-Agent LLM Systems in Banking

Multi-agent decomposition is a natural fit for banking workloads because the workloads themselves are already

organized around specialized roles: a transaction monitoring function, a fraud investigation function, a compliance sign-off, and a decision. Prior work applying LLM-based multi-agent frameworks to financial anomaly detection [1] and credit assessment [2] demonstrates that this decomposition is being actively explored in the financial domain, typically by assigning each specialized role to its own agent and coordinating them through a router or planner agent.

B. Security Risks Specific to Multi-Agent Systems

Three properties of multi-agent LLM systems create risk that does not exist in a single-agent deployment. First, agents routinely ingest content produced by other agents, not only by end users, so an attacker who compromises or spoofs one agent’s output can influence every downstream agent that consumes it [3]. Second, routing and hand-off decisions are themselves attack surfaces: an adversary that can intercept or relabel an inter-agent message can redirect a request to the wrong specialist or bypass a required review step entirely [4]. Third, tool invocation – the mechanism by which an agent actually takes action in the world, such as executing a transfer or posting a ledger entry – can be mutated in-flight, which is a materially different threat from a model simply producing an incorrect answer. A broad survey of agentic security confirms that these classes of attack generalize across frameworks and that isolated, model-level defenses are insufficient against an adversary operating at the orchestration layer [5], [6].

C. The Gap: A Reusable, Domain-Grounded Testbed

General red-teaming benchmarks for LLM agents [7] provide valuable, reusable attack scaffolding, but they evaluate agents against synthetic or generic tasks rather than an existing production-shaped business workflow, and they are not designed around the constraint that the business logic under test must remain completely unmodified. This second constraint matters in practice: a testbed that requires editing the fraud-review agent’s prompt or control flow to add an experiment cannot be trusted to measure that agent’s real behavior. MANTIS is designed specifically to satisfy both properties at once – a realistic, domain-grounded multi-agent system, instrumented non-invasively.

D. Zero Trust as a Complementary, Deferred Direction

Zero Trust architecture – the principle of never trusting a request implicitly, regardless of its origin, and mediating every action through explicit policy [8] – is increasingly being proposed as a governance layer for agentic AI systems specifically, including fine-grained, identity-based access control for individual agents [9]. MANTIS’s inventory of agents and tools, and its per-control-point metadata, are exactly the inputs a Zero Trust policy decision point would need. We treat Zero Trust enforcement as a deliberately separate, later research contribution: this paper’s scope is limited to making that inventory and control-point

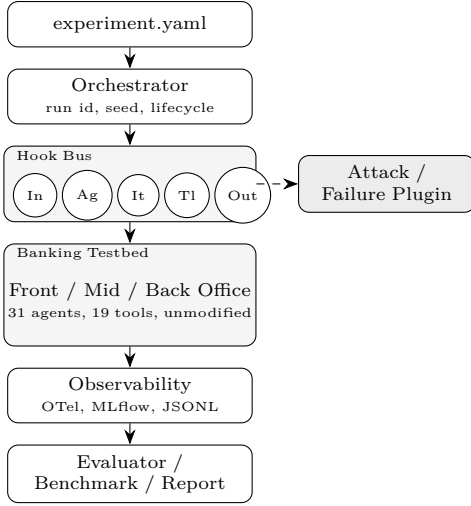


Fig. 1. MANTIS pipeline. Every run passes through the same five-point hook bus regardless of banking domain; an attack or failure plugin attaches to the bus and is dispatched alongside the standard observability plugin, without touching the banking agents beneath it.

metadata available through a documented interface, not to implementing enforcement.

III. Methodology

A. System Overview

Figure 1 shows the MANTIS pipeline. An experiment is declared entirely in a YAML configuration, which an orchestrator loads to establish a run identifier, a seed, and the experiment’s lifecycle. The orchestrator drives the banking workflow through a hook bus exposing five control points. An attack or failure plugin registers against the hook bus at one of those five points; every dispatch also passes through an observability plugin that is guaranteed to observe – though never able to reverse – whatever an attack plugin has already done to the payload in that same dispatch. Beneath the hook bus, a banking runtime adapter presents one stable interface (inventory, workflow lookup, and execution) over the front-, mid-, and back-office agents, which are otherwise unmodified. Every event is written to a standard observability pipeline, which an evaluator and benchmark runner then consume to produce scored, reproducible results.

B. Banking Runtime Adapter and Domain Coverage

The banking runtime adapter presents a single interface – `inventory()`, `workflows()`, and `build()` – over three domains. The front office covers real-time transaction monitoring, fraud detection, compliance, and a customer-service chatbot, comprising 11 agents. The mid office covers operations planning and representative-assist workflows, comprising 12 agents. The back office covers end-of-day reconciliation and settlement, comprising 8 agents. Across the three domains, agents call 19 distinct banking tools, from read operations such as retrieving a

customer’s transaction history to write operations with a direct financial side effect, such as executing a transfer or posting a ledger update. The inventory returned by the adapter is introspected live from the tool modules and the agent registry at call time, rather than maintained as a hand-authored list, so that the reported system surface cannot silently drift from the deployed system.

C. Hook Bus and Five Control Points

The hook bus dispatches a typed context object – carrying a run identifier, trace identifier, workflow identifier, the control-point stage, a source and target, a payload, and free-form metadata – to every registered plugin whose declared stages match the current control point. A plugin returns one of five actions: continue, mutate, skip, deny, or error. When a plugin mutates the payload, the mutated version is passed to the next plugin in the chain, and the fact that a mutation (or skip, deny, or error) occurred is recorded in the context metadata so that a plugin registered later in the same dispatch – notably the observability plugin – can distinguish a genuinely adversarial event from an ordinary one, rather than simply observing an already-mutated payload indistinguishable from a normal one. The five control points – input, agent, interaction, tool, and output – are chosen to bracket every externally observable transition in the agent lifecycle: raw input before any agent reasons about it, an agent’s invocation of the underlying model, the message an agent sends to or receives from that model, a tool call and its result, and the final output returned to the caller. A coverage report records, per run, how many times each of the ten before/after hook points fired and which plugins acted on each, which is itself a queryable artifact for confirming that a given experiment configuration actually exercised the control point it claimed to.

D. Attack and Failure Plugins

Four attack plugins are implemented against the same interface. Prompt injection intercepts raw input at the input control point and injects adversarial instructions before the front-office transaction monitor reasons about a transaction. Message spoofing intercepts the interaction control point – an agent’s outgoing call to the underlying model – and substitutes a fabricated message, for example a false compliance-clearance signal, into a specific agent’s context. Route confusion intercepts a routing tool call and forces execution to a different downstream agent than the one the orchestration logic selected, for example bypassing a compliance step entirely. Tool-parameter mutation intercepts a tool call directly and rewrites its arguments, for example redirecting the destination account on a funds transfer, before the call reaches the banking backend. A fifth plugin family implements ordinary reliability failures – delay, timeout, and malformed results – so that an evaluator can separate a security-caused anomaly from a routine fault. Every attack plugin declares the control

points and parameters it supports, and every experiment configuration that enables one specifies an expected effect through the evaluation schema (expected tools, forbidden tools, and expected terminal state), giving each attack a machine-readable ground truth rather than requiring manual inspection of the transcript.

E. Observability Pipeline

Every hook dispatch is mapped to one of seven typed event families: experiment (run-level start/end, configuration hash, seed, and status), workflow (start, end, and routing decisions, tagged with business domain and terminal outcome), agent (start, end, and error, with latency), interaction (message sent to, received from, or mutated en route to the underlying model), tool (call, result, and error, tagged with operation type, risk level, data sensitivity, and financial side effect), security (an `ATTACK_INJECTED` event whenever a plugin took a non-trivial action, carrying the stage, target, plugin, and observed effect), and evaluation (the score each automated evaluator assigned, with a reference to its evidence). Every event is written as a line of portable JSON so that trace analysis never depends on a specific backend, in addition to being exported as OpenTelemetry spans and as an MLflow run carrying the same configuration hash and seed for cross-referencing.

F. Evaluation and Benchmarking Framework

An automated evaluator reads a run’s trace and reports three scores: trace completeness (whether the mandatory workflow, agent, and tool events are present), tool-use correctness (whether every tool the configuration marked as expected was actually called, and no tool marked as forbidden was called), and workflow outcome (whether the terminal state the run actually reached – inferred from the last business-significant tool invoked, such as a manual-review submission or a transfer execution – matches the configuration’s expected terminal state). A benchmark runner executes a configuration a specified number of times at a specified concurrency and reports latency and throughput; a campaign command sweeps every configuration in a directory, isolates each run’s artifacts, and produces one consolidated, human-readable report scoring every experiment side by side.

G. Reproducibility

Every run manifest records a SHA-256 hash of the fully normalized configuration, the configured random seed, and, as of this work, the Python interpreter version, the installed version of every framework-critical dependency, and the source repository’s current commit, so that a reported result can always be traced back to the exact code and environment that produced it. Because the underlying agents are backed by a live LLM, an identical seed narrows but does not eliminate run-to-run variance in exact wording; evaluators are accordingly designed to score

bounded, semantic properties – which tools were called, what terminal state was reached – rather than exact text matches.

IV. Experimental Results

A. Implementation Status

All components described in Section III are implemented and have been exercised end to end against a live backend and live model calls, in addition to an offline automated test suite of 97 unit, contract, and regression tests that run in seconds without a live model or backend. This section reports preliminary, illustrative evidence from that live verification; Section IV-E describes the full statistical evaluation campaign this testbed is designed to support, which had not yet been run in full at the time of writing.

B. Preliminary, Live-Verified Evidence

Hook coverage was measured across representative front-, mid-, and back-office runs; all ten before/after hook points recorded at least one hit in every run, confirming that no control point is silently unreachable for the workflows exercised. Each of the four attack plugins was independently verified to produce its declared effect and to emit a corresponding security event: the route-confusion plugin was observed to change a run’s evaluated terminal state from the baseline’s implicit approval to `manual_review`, and the message-spoofing plugin was observed to emit an `ATTACK_INJECTED` event together with a `MESSAGE_MUTATE` interaction event correctly attributing the mutation to the targeted agent. A preliminary observability-overhead comparison over four repeated baseline runs in each of the off and full observability modes showed average end-to-end latencies within roughly one second of each other in both directions across repeated trials – that is, the difference was not consistently signed and was smaller than the run-to-run variance introduced by live model sampling, which dominates end-to-end latency at this sample size. We report this as inconclusive rather than as evidence of negligible overhead.

C. Isolating Overhead from LLM Latency

To address the confound above, we implemented a zero-latency mock model (§ IV-E discusses its scope) that an experimenter opts into per benchmark configuration, and repeated the off/full comparison with live model sampling removed entirely. The result was, itself, informative: with LLM latency eliminated, the off/full comparison remained just as noisy and just as inconclusive at four repetitions (off: 8.53s average; full: 6.68s average – backwards from what an overhead claim would predict). This shows that our initial hypothesis was incomplete. LLM sampling latency is not the only, or even the dominant, confound at this sample size: our benchmark methodology spawns one fresh operating-system process per run to guarantee

clean process state, and that process’s startup and session-bootstrap cost – interpreter start, agent-graph construction, and MCP session establishment – itself appears to dominate wall-clock latency once model sampling is removed. A meaningful overhead measurement therefore needs to isolate instrumentation cost specifically, which requires either amortizing process startup across many repetitions per process or measuring hook-dispatch cost directly rather than end-to-end process latency; we did not do either in this preliminary pass.

D. Volume Scaling

We also implemented a scaling mode in which the benchmark runner executes a configurable list of repetition counts at a fixed concurrency, rather than a single repetition count, so throughput and latency can be reported as a function of transaction volume instead of a single point estimate. Run with the mock model at concurrency 2 across 1, 2, 4, and 8 repetitions, throughput increased monotonically (0.110, 0.227, 0.267, and 0.301 runs/s respectively) as fixed per-batch costs were amortized over more transactions – a sensible, interpretable curve, though generated with the mock model and at a small number of volume levels, not yet with a live model or across all three banking domains.

E. Remaining Work Toward a Full Evaluation Campaign

Two further extensions remain before the results in this section could be reported as a statistically established evaluation, rather than preliminary, illustrative evidence. First, baseline-versus-attack comparisons need to be run with a fixed configuration repeated across many trials for every combination of banking domain and attack family, so that reported effects carry a mean and a variance rather than a single observation. Second, isolating instrumentation overhead specifically – as opposed to the process-bootstrap-dominated measurement in § IV-C – needs a benchmark design that amortizes or removes per-process startup cost, for example by executing repeated in-process trials rather than one operating-system process per trial. We note for completeness that our mock model only emits text responses and never a function call, so it does not exercise the tool control point; a tool-call-capable mock, needed for a complete instrumentation-only benchmark, was out of scope for this preliminary pass. We report the current state of each measurement rather than a projected final result, because presenting a small number of process-startup-confounded trials as though they were a statistically established finding would overstate what has actually been shown.

V. Conclusion

We presented MANTIS, a configuration-driven testbed that instruments a realistic, unmodified banking multi-agent system at five fixed control points, so that adversarial and failure experiments are declared in YAML

rather than implemented as changes to business logic. The testbed’s banking runtime adapter, hook bus, four attack plugins, observability pipeline, and evaluation and benchmarking framework are implemented and verified end to end, including a reproducibility mechanism that now records the environment, not only the configuration, behind every run. Preliminary, live-verified evidence confirms that every control point is reachable and that every attack plugin produces and reports its declared effect. A zero-latency mock model and a volume-scaling benchmark mode, both implemented and verified live, show that our original hypothesis about the dominant source of measurement noise was incomplete: removing LLM sampling latency did not resolve the observability-overhead comparison’s noise, revealing that per-process startup and session-bootstrap cost, not instrumentation or model sampling, dominates at the sample sizes tested. A full statistical evaluation campaign – amortizing that process-bootstrap cost, running many repeated trials per domain and attack family, and extending the mock model to cover the tool control point – remains as the concrete next step toward a statistically established result. Future work extends this testbed along the directions its interfaces were explicitly designed to support without modification: a minimal schema-driven UI over the existing CLI, additional observability exporters, security-mechanism (guardrail and policy) plugins evaluated through the same plugin interface as the attacks in this paper, and a Zero Trust Backplane that consumes the same application inventory and control-point metadata described in Section III to generate and enforce access policy as a separate research contribution.

Acknowledgment

References

- [1] X. Zhang et al., “Enhancing Anomaly Detection in Financial Markets with an LLM-based Multi-Agent Framework,” arXiv preprint arXiv:2403.19735, 2024.
- [2] “MASCA: LLM based-Multi Agents System for Credit Assessment,” arXiv preprint arXiv:2507.22758, 2025.
- [3] D. Lee and M. Tiwari, “Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems,” arXiv preprint arXiv:2410.07283, 2024.
- [4] “Red-Teaming LLM Multi-Agent Systems via Communication Attacks,” in Findings of the Association for Computational Linguistics: ACL 2025, arXiv preprint arXiv:2502.14847.
- [5] “A Survey on Agentic Security: Applications, Threats and Defenses,” arXiv preprint arXiv:2510.06445, 2025.
- [6] “Security Considerations for Multi-agent Systems,” arXiv preprint arXiv:2603.09002, 2026.
- [7] “REDAgentBench: Executable Red Teaming and Faithful Measurement of LLM Agent Systems,” arXiv preprint arXiv:2608.10669, 2026.
- [8] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020.
- [9] “A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control,” arXiv preprint arXiv:2505.19301, 2025.